

Senkronizasyon: Mutex, Semafor, Üretici-Tüketici

İşletim Sistemleri — Konu 4

1. Neden Senkronizasyon Gerekir?

Aynı sürece ait thread'ler adres uzayını paylaşır (Konu 1). Birden fazla thread aynı paylaşımlı veriyi eşzamanlı okuyup yazarsa bir **yarış durumu (race condition)** oluşur: sonuç, thread'lerin zamanlamasına (hangi context switch ne zaman olduğuna) bağlı hale gelir ve deterministik olmaktan çıkar.

Paylaşımlı veriye erişen ve yalnız bir thread'in aynı anda içinde olabileceği kod bloğuna **kritik bölge (critical section)** denir. Bir kritik bölge çözümü şu üç özelliği sağlamalıdır:

1. **Karşılıklı dışlama (mutual exclusion)**: Aynı anda en fazla bir thread kritik bölgede olabilir.
2. **İlerleme (progress)**: Kritik bölgede kimse yokken, girmek isteyenler arasından seçim sonsuza kadar ertelenemez.
3. **Sınırlı bekleme (bounded waiting)**: Bir thread'in kritik bölgeye girme isteği sonsuza kadar ertelenemez (açlık olmamalı).

2. Mutex (Karşılıklı Dışlama Kilidi)

Mutex, en fazla bir thread'in "sahip" olabildiği ikili bir kilittir:

```
mutex.lock()      // kritik bölgeye giriş; kilit doluysa bloklanır
                // kritik bölge
mutex.unlock()    // kritik bölgeden çıkış
```

Kritik kural: mutex'i **kilitleyen thread onu açmalıdır**. Başka bir thread'in kilidini açmaya çalışması tanımsız davranıştır. Mutex, kavramsal olarak değeri yalnız 0 veya 1 olan bir semafor gibi düşünülebilir, ama sahiplik (ownership) semantiği taşır.

3. Semafor (Semaphore)

Bir **semafor**, negatif olmayan bir tam sayı sayaç ve iki atomik işlemdir:

- **wait()** (bazı kaynaklarda **P()** veya **acquire()**): sayaç > 0 olana kadar bekler, sonra sayacı 1 azaltır.

- **signal()** (bazı kaynaklarda `V()` veya `release()`): sayacı 1 artırır ve bekleyen bir thread varsa uyandırır.

İkili semafor (binary semaphore), mutex'e benzer ama sahiplik zorlaması yoktur — `signal()`'i başka bir thread çağırabilir; bu da onu **sayan semafor (counting semaphore)** ile birlikte üretici-tüketici gibi problemlerde sinyal göndermek için kullanışlı yapar.

4. Üretici-Tüketici Problemi (Producer-Consumer)

Sabit kapasiteli bir tampon (buffer) etrafında iki thread çalışır: **üretici** tampona veri ekler, **tüketici** tampondan veri alır. Klasik çözüm üç senkronizasyon nesnesi kullanır:

- mutex: tampona erişimi (ekleme/çıkarma index'lerini) korur — kritik bölge.
- empty (sayan semafor, başlangıç = tampon kapasitesi): boş yuva sayısı.
- full (sayan semafor, başlangıç = 0): dolu yuva sayısı.

Doğru sıra üretici tarafında:

```
wait(empty)      // önce boş yuva var mı diye sinyal semaforunu düşür
mutex.lock()
    tampona_ekle(item)
mutex.unlock()
signal(full)     // sonra "bir öge daha var" diye tüketiciye haber ver
```

Doğru sıra tüketici tarafında:

```
wait(full)       // önce dolu yuva var mı diye sinyal semaforunu düşür
mutex.lock()
    item = tampondan_al()
mutex.unlock()
signal(empty)    // sonra "bir yuva boşaldı" diye üreticiye haber ver
```

Kritik nokta — sıra neden önemli: `wait(empty)/wait(full)` çağrıları **mutex dışında ve ondan önce** yapılır. Sinyal semaforu beklemesi mutex kilitliken yapılırsa (yanlış sıra), tampon doluyken bekleyen üretici mutex'i tutar durumda kilitlenmiş kalabilir ve tüketici mutex'i hiç alamadığı için tamponu boşaltamaz — sistem **kilitlenir (deadlock)**. Bu yanlış sıralama, `sample_data/isletim-sistemleri/producer_consumer.py` dosyasındaki hatalı örnekte kasıtlı olarak gösterilmiştir (30 koşumluk deneyle doğrulandı: 30/30 deadlock, başka sonuç yok). Tamponun taşması (overflow) veya taşınması (underflow), bu koddaki hatadan DEĞİL, `signal(full)`'ün tampona ekleme işleminden önce çağırılması gibi ayrı bir hatadan kaynaklanır — burada karıştırılmamalı.

5. Mutex mi, Semafor mu?

	Mutex	Semafor
Amaç	Karşılıklı dışlama (kritik bölge)	Sinyalleşme + karşılıklı dışlama
Sahiplik	Var — kilitleyen açar	Yok — herhangi bir thread signal çağırabilir
Değer aralığı	0/1	0..N (sayan) veya 0/1 (ikili)
Tipik kullanım	Paylaşımlı veri koruma	Üretici-tüketici, kaynak havuzu sayımı

6. Özet

- Kritik bölge: karşılıklı dışlama + ilerleme + sınırlı bekleme sağlanmalı.
- Mutex sahiplikli bir kilittir; semafor sayaç tabanlı, sahipliksiz bir senkronizasyon aracıdır ve sinyalleşme için de kullanılabilir.
- Üretici-tüketici probleminde `wait()/signal()` sırası kritiktir: sinyal semaforu beklemesi her zaman mutex'ten ÖNCE ve dışında yapılır; aksi halde deadlock riski doğar (bkz. `producer_consumer.py`'deki hatalı örnek).